



(12) 发明专利

(10) 授权公告号 CN 111597070 B

(45) 授权公告日 2020. 11. 27

(21) 申请号 202010727337.8

审查员 张琳琳

(22) 申请日 2020.07.27

(65) 同一申请的已公布的文献号

申请公布号 CN 111597070 A

(43) 申请公布日 2020.08.28

(73) 专利权人 北京必示科技有限公司

地址 100083 北京市海淀区中关村东路8号
东升大厦A座4层

(72) 发明人 李则言 张文池 程博 黄成

陈哲康 沈梦家 隋楷心 刘大鹏

(74) 专利代理机构 北京华创智道知识产权代理

事务所(普通合伙) 11888

代理人 彭随丽

(51) Int. Cl.

G06F 11/07 (2006.01)

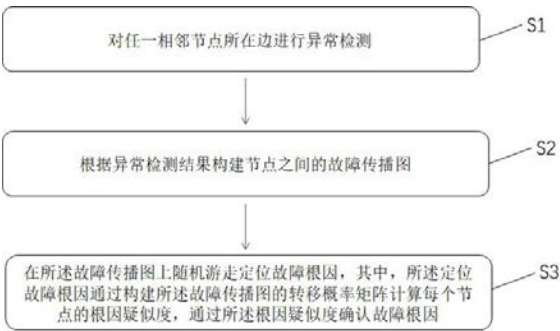
权利要求书1页 说明书8页 附图4页

(54) 发明名称

一种故障定位方法、装置、电子设备及存储介质

(57) 摘要

本发明涉及计算机技术领域,公开了一种故障定位方法、装置、电子设备及存储介质,其中方法包括以下步骤:对任一相邻节点所在边进行异常检测;根据异常检测结果构建节点之间的故障传播图;在所述故障传播图上随机游走定位故障根因,其中,所述定位故障根因通过构建所述故障传播图的转移概率矩阵计算每个节点的根因疑似度,通过所述根因疑似度确认故障根因。本发明以无监督的方式,综合多个指标,通过服务之间指标异常对服务整体异常的影响力,定量分析故障在服务之间的传播关系,基于此通过随机游走算法判断根因,为一套有效的自动化定位故障根因服务的方案。



1. 一种故障定位方法,其特征在于,所述方法包括:

对任一相邻节点所在边进行异常检测;所述异常检测包括如下步骤:对任一相邻节点所在边的所有指标进行异常度计算;选择异常度最大值作为该边的异常度;通过所述异常度的阈值判断每一边是否异常;

根据异常检测结果构建节点之间的故障传播图;根据当前的异常指标自动确定故障传播方向,所述故障传播方向从调用方到服务方或者从服务方到调用方;

在所述故障传播图上随机游走定位故障根因,其中,所述定位故障根因通过相邻节点所在边的异常度构建所述故障传播图的转移概率矩阵计算每个节点的根因疑似度,通过所述根因疑似度确认故障根因。

2. 如权利要求1所述的故障定位方法,其特征在于,所述异常度计算通过指标的观测值、均值和标准差计算得到。

3. 如权利要求1所述的故障定位方法,其特征在于,所述随机游走包括如下步骤:

为所述故障传播图的每个节点添加自环,所述自环的转移概率由该节点与异常度最大的后节点和异常度最大的前节点的异常度差值决定。

4. 如权利要求3所述的故障定位方法,其特征在于,所述随机游走包括反向游走。

5. 如权利要求1所述的故障定位方法,其特征在于,任一所述节点为故障根因的疑似度由转移概率矩阵和该节点的异常度向量决定。

6. 一种故障定位装置,其特征在于,包括:

异常检测模块,用于对任一相邻节点所在边进行异常检测;所述异常检测包括如下步骤:对任一相邻节点所在边的所有指标进行异常度计算;选择异常度最大值作为该边的异常度;通过所述异常度的阈值判断每一边是否异常;

构建故障传播图模块,用于根据异常检测结果构建节点之间的故障传播图;根据当前的异常指标自动确定故障传播方向,所述故障传播方向从调用方到服务方或者从服务方到调用方;

以及根因定位模块,在所述故障传播图上随机游走定位故障根因,其中,所述定位故障根因通过相邻节点所在边的异常度构建所述故障传播图的转移概率矩阵计算每个节点的根因疑似度,通过所述根因疑似度确认故障根因。

7. 一种电子设备,其特征在于,包括:

至少一个处理器;以及

与所述至少一个处理器耦合连接的存储器;其中,

所述存储器存储有计算机程序,所述计算机程序能够被所述至少一个处理器执行,以实现权利要求1-5任一项所述的方法。

8. 一种计算机可读存储介质,其特征在于,所述计算机可读存储介质用于存储计算机程序,当所述计算机程序被执行时,能够实现权利要求1-5任一项所述的方法。

一种故障定位方法、装置、电子设备及存储介质

技术领域

[0001] 本发明涉及计算机技术领域,更具体地说,涉及一种故障定位方法、装置、电子设备及存储介质。

背景技术

[0002] 因为在交付、扩容和自动化等方面的优势,基于服务的系统架构在大型服务的设计中被越来越多地采用。一个基于服务的系统通常可以具有几十个到几百个的服务,它们部署于成百上千的服务器上。虽然有很多技术应用于这类系统用来保证起质量,但是其中的错误依然是不可避免的。系统的故障会带来极其巨大的经济损失。

[0003] 通常,在每个服务上会部署众多的指标收集器(例如CPU使用率,服务响应率,成功率等)和异常检测器,它们被用来检测服务的异常。但是,人工分析系统的故障依然是非常有挑战的。因为在基于服务的系统上,为了完成一个用户的请求,需要许多服务通过相互调用的形式配合实现。因此当一个服务发生故障的时候,会有众多的服务因为依赖关系而也产生异常,发出告警。此时运维人员必须手动逐个查看这些异常的服务,排除掉其中仅仅只是受影响而没有故障的服务,最终定位到根因服务。对于大型系统而言,不同的服务可能由不同的运维人员甚至不同的部门管理,所以一次故障会涉及到众多的运维人员和部门参与,定位成本极高。

[0004] 为了快速定位故障根因,自动化的定位算法是非常必要的。本发明的发明人发现实现自动化定位有以下几个难点:首先,服务之间有复杂的依赖关系。其次,基于服务的系统迭代频率高,经常会进行变更。最后,这类系统上有着海量的指标,和故障有关的指标就会被淹没在海量的指标中。

[0005] 现有方法,需要通过许多过去积累的故障数据和对应的根因,即需要人工标注的标注数据,从中学习才能得到模型,进而定位。

发明内容

[0006] 本发明的目的是解决上述提出的问题,提出了一种快速、稳健的故障定位方法,本方案包括:

[0007] 对任一相邻节点所在边进行异常检测;

[0008] 根据异常检测结果构建节点之间的故障传播图;

[0009] 在所述故障传播图上随机游走定位故障根因,其中,所述定位故障根因通过构建所述故障传播图的转移概率矩阵计算每个节点的根因疑似度,通过所述根因疑似度确认故障根因。

[0010] 优选的,所述异常检测包括如下步骤:

[0011] 对任一相邻节点所在边的所有指标进行异常度计算;

[0012] 选择异常度最大值作为该边的异常度;

[0013] 通过异常度的阈值判断每一边是否异常。

- [0014] 优选的,所述异常度计算基于3-sigma算法;
- [0015] 优选的,所述异常度计算通过指标的观测值、均值和标准差得到。
- [0016] 优选的,所述构建节点之间的故障传播图的规则为:
- [0017] 若节点关系仅为调用关系,根据当前的异常指标自动确定故障传播方向,所述故障传播方向与所述故障传播图的游走方向相反;
- [0018] 若节点关系存在资源共享关系,故障从一个节点转移到非调用关系的另一个节点;
- [0019] 优选的,所述随机游走的步骤包括:
- [0020] 为所述故障传播图的每个节点添加自环,所述自环的转移概率由该节点与异常度最大的后节点和异常度最大的前节点的异常度差值决定;
- [0021] 优选的,所述随机游走包括反向游走。
- [0022] 优选的,任一所述节点为故障根因的疑似度由转移概率矩阵和该节点的异常度向量决定。
- [0023] 优选的,所述故障定位方法包括:在异常检测之前通过数据预处理,去除离群点。
- [0024] 离群点指的是指标中的点,一个指标是很多个点组成的,我们使用指标历史数据(即故障发生前的数据点)检测异常,为避免影响异常检测,去除历史数据里的异常数据点(离群点)。
- [0025] 本发明的另一方面,提供一种故障定位装置,包括:
- [0026] 异常检测模块,用于对任一相邻节点所在边进行异常检测;
- [0027] 构建故障传播图模块,用于根据异常检测结果构建节点之间的故障传播图;以及
- [0028] 根因定位模块,在所述故障传播图上随机游走定位故障根因,其中,所述随机游走通过构建所述故障传播图的邻接矩阵计算每个节点的根因疑似度,通过所述根因疑似度确认故障根因。
- [0029] 优选的,所述异常检测模块包括:
- [0030] 异常计算子模块,用于对任一相邻节点所在边的所有指标进行异常计算;
- [0031] 选择子模块,用于选择异常度最大值作为该边的异常度;
- [0032] 异常确定子模块,通过异常度的阈值确定每一边是否异常。
- [0033] 优选的,所述异常计算基于3-sigma算法;
- [0034] 优选的,所述异常度计算基于指标的观测值、均值和标准差计算得到。
- [0035] 优选的,所述构建故障传播图模块的规则为:
- [0036] 若节点关系仅为调用关系,根据当前的异常指标自动确定故障传播方向,所述故障传播方向与所述故障传播图的游走方向相反;
- [0037] 若节点关系存在资源共享关系,故障从一个节点转移到非调用关系的另一个节点。
- [0038] 优选的,所述根因定位模块包括:
- [0039] 添加自环子模块,用于为所述故障传播图的每个节点添加自环,所述自环的转移概率由该节点的后节点和前节点的异常度差值决定。
- [0040] 优选的,所述随机游走包括反向游走。
- [0041] 优选的,任一所述节点为故障根因的疑似度由转移概率矩阵和该节点的异常度向

量决定。

[0042] 本发明的另一方面在于,提供一种电子设备,包括:

[0043] 至少一个处理器;以及

[0044] 与所述至少一个处理器耦合连接的存储器;其中,

[0045] 所述存储器存储有计算机程序,所述计算机程序能够被所述至少一个处理器执行,以实现本发明所述的方法。

[0046] 本发明的另一方面在于,提供一种计算机可读存储介质,所述计算机可读存储介质用于存储计算机程序,当所述计算机程序被执行时,能够实现本发明所述的方法。

[0047] 有益效果:

[0048] 本发明使用基于随机游走的算法,成功建模了故障通过负载的依赖关系传导的过程,解决了现有技术中针对服务间依赖关系复杂的问题。

[0049] 本发明是一个无参数化的定位算法,当故障发生、本算法被触发之后,只需要利用触发后获取的信息即可进行定位,不需要预训练的模型。针对任何故障,本发明的算法都是基于系统最新的状态直接定位的,而不基于(可能已过时的)预训练的模型。因此本发明可以应对快速迭代的系统。

[0050] 本发明使用一种快速、稳健的业务指标分析方法,可以快速处理大量指标和正确处理多种类型的指标,作为根因定位的基础,归纳了不同情况下故障传播方向的定性规则;以无监督的方式,综合多个指标,针对一条边有很多指标的情况,我们综合多指标给出一条边的异常程度和故障传播的判断,通过服务之间指标异常对服务整体异常的影响力,定量分析故障在服务之间的传播关系,基于此通过随机游走算法判断根因,为一套有效的自动化定位故障根因服务的方案。

[0051] 传统的基于阈值的检测方法不能自动适应监控指标在时间和空间上的剧烈变化,本方案通过采用较常用的高斯分布假设,通过3-sigma规则计算异常度,实现了一种稳健快速的异常检测方法。

[0052] 针对传统方法不能对多指标进行综合定位的问题,本方案使用无监督的异常检测算法,未使用到任何预先人工方法标注出来的标注数据,综合多指标的异常检测和根因定位方法。在基于服务的系统中,故障在不同的微服务上可能表现在不同的指标上,因此基于多指标的定位是必要的。本方案通过多指标的综合定位解决了异常在不同指标间传递的问题。

附图说明

[0053] 图1是本发明实施例1中故障定位方法的流程图;

[0054] 图2是本发明实施例1中故障定位方法的步骤S1的步骤流程图;

[0055] 图3是本发明实施例1中构建节点之间的故障传播图的规则图;

[0056] 图4是本发明实施例1中故障定位装置的结构图;

[0057] 图5是本发明实施例1中异常检测模块10的结构图;

[0058] 图6是本发明实施例2中故障定位方法的3-sigma算法示意图;

[0059] 图7是本发明实施例2中故障定位方法随机游走陷阱图。

具体实施方式

[0060] 下面将参照附图更详细地描述本发明的具体实施例。虽然附图中显示了本发明的具体实施例,然而应当理解,可以以各种形式实现本发明而不应被这里阐述的实施例所限制。相反,提供这些实施例是为了能够更透彻地理解本发明,并且能够将本发明的范围完整的传达给本领域的技术人员。

[0061] 本发明主要分为异常检测,构建故障传播图和随机游走三个步骤。首先,对故障时段的服务之间的KPI进行异常检测。然后,我们构建故障在服务之间的故障传播图。最后,在图上应用随机游走算法,在故障传播图上定位到根因。

[0062] 实施例1本实施例提供一种故障定位方法

[0063] 如图1所示,本实施例提供的方法包括:

[0064] S1对节点之间的KPI进行异常检测;

[0065] S2根据异常检测结果构建节点之间的故障传播图;

[0066] S3在所述故障传播图上随机游走定位故障根因,其中,所述定位故障根因通过构建所述故障传播图的转移概率矩阵计算每个节点的根因疑似度,通过根因疑似度确认故障根因。

[0067] 如图2所示,S1中异常检测包括如下步骤:

[0068] S101任一相邻节点所在边的所有指标进行异常度计算;

[0069] S102选择异常度最大值作为该边的异常度;

[0070] S103通过异常度的阈值判断每一边是否异常。

[0071] 所述异常度计算基于3-sigma算法,通过指标的观测值、均值和标准差得到。

[0072] 随机游走的步骤包括:

[0073] 为所述故障传播图的每个节点添加自环,所述自环的转移概率由该节点与异常度最大的后节点和异常度最大的前节点的异常度差值决定。

[0074] 在优选的实施例中,随机游走包括反向游走。在一次随机游走中,如果算法落入了不可能是根因的分支,那么就没有机会退出,只能沿着错误的分支继续下去。因此算法为每条边都增加了反向边。

[0075] 在优选的实施例中,如图3所示,构建节点之间的故障传播图的规则为:

[0076] 若节点关系仅为调用关系,根据当前的异常指标自动确定故障传播方向,所述故障传播方向与所述故障传播图的游走方向相反;

[0077] 若节点关系存在资源共享关系,故障从一个节点转移到非调用关系的另一个节点。

[0078] 在优选的实施例中,任一所述节点为故障根因的疑似度由转移概率矩阵和该节点的异常度向量决定。

[0079] 本实施例的另一方面,提供一种故障定位装置,如图4所示,包括:

[0080] 异常检测模块10,用于对节点之间的KPI进行异常检测;

[0081] 构建故障传播图模块20,用于根据异常检测结果构建节点之间的故障传播图;以及

[0082] 根因定位模块30,在所述故障传播图上随机游走定位故障根因,其中,所述随机游走通过构建所述故障传播图的邻接矩阵计算每个节点的根因疑似度,通过根因疑似度的大

小确认故障根因。

[0083] 如图5所示,异常检测模块10包括:

[0084] 异常计算子模块101,用于对每一节点的每一指标进行异常计算;

[0085] 选择子模块102,用于选择异常度最大值作为每一节点的异常度;

[0086] 异常确定子模块103,通过异常度的阈值确定每一节点是否异常。

[0087] 异常计算基于3-sigma算法,通过指标的观测值、均值和标准差计算得到。

[0088] 根因定位模块30包括:

[0089] 添加自环子模块,用于为所述故障传播图的每个节点添加自环,所述自环的转移概率由该节点的后节点和前节点的异常度差值决定;

[0090] 本实施例的另一方面提供一种电子设备,包括:

[0091] 至少一个处理器;以及

[0092] 与所述至少一个处理器耦合连接的存储器;其中,

[0093] 所述存储器存储有计算机程序,所述计算机程序能够被所述至少一个处理器执行,以实现本发明所述的方法。

[0094] 本实施例的另一方面在于,提供一种计算机可读存储介质,所述计算机可读存储介质用于存储计算机程序,当所述计算机程序被执行时,能够实现本发明所述的方法。

[0095] 传统的根因定位方法只是沿着调用关系,从调用方往服务方逐渐追溯,找到最底层的异常服务方即认为是根因。但是,在不同的情况下,调用的异常可能由发起方也可能是由服务方导致的。基于对实际情况的分析,本方案实现了一个基于规则的故障根因传播方向判断方法,帮助后续的根本定位从而能够定位到更多类型的异常的根因,更加普适。

[0096] 实施例2 本实施例提供另一种故障定位方法

[0097] 1. 异常检测

[0098] 如图6所示,为3-sigma算法示意图,其中A,B,C,D,E是几个服务节点,箭头表示节点之间的调用关系,需要对图上每一条边进行异常检测,每一条边包含若干个指标,例如总交易量,成功率,响应率,平均响应时间等,对每一个指标分别进行异常检测。

[0099] 本实施例使用了常用的3-sigma算法进行异常检测。具体做法是,对每一条边的每个指标,我们都计算其异常度,然后选择所有指标异常度的最大值作为一条边的异常度。最后我们通过异常度的阈值确定每一条边是否异常。3-sigma算法如下:

$$[0100] \quad A(ij, m) = \frac{|\mu_t - v_t|}{\sigma_t}$$

[0101] 其中A(ij, m)代表t时刻任一边的任一指标的异常度,ij指代任一边,i和j指代任一边的两节点, v_t 是t时刻任一指标的观测值, μ_t , σ_t 分别是t时刻任一指标对应的均值和标准差,均值和标准差通过过去若干个周期同一相位的数据计算,例如当前时刻是周一13:00,那么取的数据就是过去3周每周周一的13:00附近的数据,或者过去7天每天13:00附近的数据。异常度的阈值为用户人工指定,默认为3。

[0102] 我们使用历史数据计算均值和标准差,进而进行异常检测,但是历史数据中可能本身存在异常点(离群点),会影响异常检测的效果。

[0103] 为了进一步提升算法的稳健型,避免在历史数据中的离群点,或者异常数据点影响异常检测,在计算 μ_t , σ_t 之前,会事先去掉历史数据中的最小的5%和最大的5%的数据点。

这样在一般情况下,留下的历史数据点将只包含正常的历史数据,因此本异常检测方法能够适应数据中的噪声和更早时刻的异常点,更加稳健。

[0104] 2. 构建故障传播图

[0105] 随机游走算法认为,在当前游走到于某个异常节点(即服务)的时候,继续根据相邻节点是根因的概率选择下一步游走的目标。这样最终多次游走最终收敛到的目标就是根因。而故障可能有不同的传播方向。

[0106] 例如,如果故障是因为某服务所在的物理机硬件故障导致无响应,那么异常会从目的系统传播到源系统,此时响应率指标是异常的,但是总交易量是正常的或者变低。如果故障是由于某服务发出的请求过大导致后续被调用服务无响应,那么异常会从服务方传播到调用方,此时只有总交易量指标是异常升高的,但是其他指标比如响应时间可能是正常的。

[0107] 算法根据当前的异常指标自动确定故障的传播方向,从而得到故障传播图。故障从源节点传播到目的节点的时候,故障传播图上的边的方向是从目的节点到源节点;故障从目的节点传播到源节点的时候,故障传播图上的边的方向是用源节点到目的节点。

[0108] 另一方面,节点之间除了调用关系,还存在资源共享的关系。例如,可能多个服务共享一个物理机,那么当一个服务交易量过大引起CPU过载的时候,自然会影响到另一个服务。因此我们还在所有的共享资源的服务之间增加双向的边,变得权重不是异常度,而是服务之间的相关关系。即,如果两个服务共享资源,而且它们的指标之间相关性大,那么故障就可能从一个服务转移到另一个服务。

[0109] 到这一步得到了一个故障传播图,形式化地描述为 $G=(V,E)$,其中 $V=S_1, S_2, \dots, S_N$ 为N个节点的集合; $E=(S_i, S_j, A_{ij})$,其中E指代边的集合, S_i, S_j 是任意两个相邻节点, A_{ij} 是边的异常度,这三个值描述了一条边。边的方向不表示调用关系,而是由上一步故障传播方向确定的。 $A(N \times N)$ 是故障传播图的邻接矩阵,带有权重的异常度矩阵。

[0110] 根据算法的设计思路,若我们从一个节点游走到另一个节点,等价于认为新节点是当前节点的异常根因。但是,如果当前节点已经很可能是根因,传统的随机游走算法还是会强制选择一个邻居继续游走下去。为了避免这种情况,每个节点会加入一条自环,自环为图论中的术语,代表每个节点到自己节点的形成的封闭环,自环对应的转移概率由该节点的前驱和后继节点的异常度之差决定,其中前驱节点代表该节点与所有相邻的前节点,计算该节点与所有相邻的前节点的异常度,选择其中异常度最大的作为前驱节点异常度,后继节点代表该节点与所有相邻的后继节点,计算该节点与所有相邻的后节点的异常度,选择其中异常度最大的作为后继节点异常度,如果该节点的后继节点比前驱节点更异常,那么说明后继节点可能是根因,那么应该以更大的概率继续游走,否则不应该继续游走。因此通过自环表达自己就是根因的可能性。

[0111] 在一次随机游走中,如果算法落入了不可能是根因的分支,那么就没有机会退出,如图7所示为随机游走陷阱图,当算法从 v_1 游走到 v_3 后,只能沿着错误的分支继续下去,再也没有机会访问 v_2 ,因此算法为每条边都增加了反向边。通过反向边实现算法的容错性。

[0112] 因此,改进的故障传播图的邻接矩阵如下:

$$[0113] \quad A'_{ij} = \begin{cases} A_{ij} + \rho A_{ji} & j \neq i \\ \max(0, \max_k A_{ki} - \max_l A_{jl}) & i = j \end{cases}$$

[0114] 式中 j, l, k 代表节点编号, A_{ki}, A_{ij}, A_{jl} 分别代表节点所在边的异常度, 如 A_{ki} 代表节点 S_k 到节点 S_i 的边的异常度, 如果从 S_i 到 S_j 的调用边的故障转移方向是从 S_j 到 S_i (从服务方到调用方), 那么 A_{ij} 是这条边的异常度。如果从 S_i 到 S_j 的调用边的故障转移方向是从 S_i 到 S_j (从调用方到服务方), 那么 A_{ji} 是这条边的异常度。当 i 不等于 j 的时候, 给每条边添加反向边, 如上述公式中的 $A_{ij} + \rho A_{ji}$, 为反向公式, 反向边公式中的 ρ 表示反向边的权重, 故障传播图越可靠, ρ 相应设置得越小, 一般取经验值0.1。当 i 等于 j 的时候, 如上式中的 $\max(0, \max_k A_{ki} - \max_l A_{jl})$, 为自环公式, $\max_k A_{ki}$ 表示节点 i 的所有入边的异常度的最大值, $\max_l A_{jl}$ 表示节点 j 的所有出边的转移概率的最大值。这两个值的差就是自环的异常度, 自环的最小值被限制为0。在具体算法中, 以转移概率的大小代表异常度大小, 即以转移概率的大小判断本节点为根因的可能性。

[0115] 对该矩阵做归一化, 得到转移概率矩阵 Q_{ij} , 其表达式如下:

$$[0116] \quad Q_{ij} = \frac{A_{ij}}{\sum_k A_{ik}}$$

[0117] 记 $\pi \in (0, 1]^{(1 \times N)}$ 是PPV (personalized PageRank vector, 个性化PageRank向量), u 为每个节点的异常度向量, $u = 1/N * A1$, 其中 A 是异常度矩阵, 1 是向量 1 (每个元素都是1的向量), $A1$ 这里指的是矩阵 A 和向量 1 做矩阵乘法。具体的物理含义就是对 A 的每一行求和。那么PPV通过如下迭代得到:

$$[0118] \quad \pi^n = \alpha \pi^{(n-1)} Q_{ij} + (1-\alpha) u$$

[0119] 其中 α 为经验值, $\pi^{(n-1)}$ 和 $\pi^{(n)}$ 分别代表 n 次迭代和 $n-1$ 次迭代, 取值范围控制随机游走和远距离随机游走的平衡。当故障传播图越可靠的时候, α 可以设置得更大。一般取默认值 $\alpha=0.85$ 即可。只要 Q_{ij} 是合法的转移概率矩阵, 上述迭代式就能收敛。实践中只需要迭代到前后两次迭代的结果变化小于给定的阈值即可。

[0120] 每个节点是根因的疑似度为 π_i , 因此算法最后输出 $\text{reverersed}(\text{argsort}(\pi))$ 。 π_i 是节点 S_i 的根因疑似度, 这里 $\text{reverersed}(\text{argsort})$ 指的是按 π_i 的降序对所有节点进行排序, 输出排序靠前的节点编号作为根因。

[0121] 通过定位算法中转移概率、自环和反向边的定义, 通过随机游走的迭代方法完成对故障传播关系的分析, 高效计算每个节点的根因疑似度。

[0122] 在基于服务的系统中, 故障在不同的微服务上可能表现在不同的指标上, 因此基于多指标的定位是必要的。本方案通过多指标的综合定位解决了异常在不同指标间传递的问题。本领域普通技术人员可以意识到, 结合本文中所公开的实施例描述的各示例的模块及算法步骤, 能够以电子硬件、或者计算机软件和电子硬件的结合来实现。这些功能究竟以硬件还是软件方式来执行, 取决于技术方案的特定应用和设计约束条件。专业技术人员可以对每个特定的应用来使用不同方法来实现所描述的功能, 但是这种实现不应认为超出本发明的范围。

[0123] 所属领域的技术人员可以清楚地了解到, 为描述的方便和简洁, 上述描述的装置和设备的具体工作过程, 可以参考前述方法实施例中的对应过程, 在此不再赘述。

[0124] 在本申请所提供的实施例中, 应该理解到, 所揭露的装置和方法, 可以通过其它的方式实现。例如, 以上所描述的装置实施例仅仅是示意性的, 例如, 所述模块的划分, 仅仅为一种逻辑功能划分, 实际实现时可以有另外的划分方式, 例如多个模块或组件可以结合或

者可以集成到另一个系统,或一些特征可以忽略,或不执行。另一点,所显示或讨论的相互之间的耦合或直接耦合或通信连接可以是通过一些接口,装置或模块的间接耦合或通信连接,可以是电性,机械或其它的形式。

[0125] 所述作为分离部件说明的模块可以是或者也可以不是物理上分开的,作为模块显示的部件可以是或者也可以不是物理模块,即可以位于一个地方,或者也可以分布到多个网络模块上。可以根据实际的需要选择其中的部分或者全部模块来实现本发明实施例方案的目的。

[0126] 另外,在本发明各个实施例中的各功能模块可以集成在一个处理模块中,也可以是各个模块单独物理存在,也可以两个或两个以上模块集成在一个模块中。

[0127] 所述功能如果以软件功能模块的形式实现并作为独立的产品销售或使用,可以存储在一个计算机可读取存储介质中。基于这样的理解,本发明的技术方案本质上或者说对现有技术做出贡献的部分或者该技术方案的部分可以以软件产品的形式体现出来,该计算机软件产品存储在一个存储介质中,包括若干指令用以使得一台计算机设备(可以是个人计算机,服务器,或者网络设备)执行本发明各个实施例节能信号发送/接收的方法的全部或部分步骤。而前述的存储介质包括:U盘、移动硬盘、ROM、RAM、磁碟或者光盘等各种可以存储程序代码的介质。

[0128] 以上描述仅为本申请的较佳实施例以及对所运用技术原理的说明。本领域技术人员应当理解,本申请中所涉及的发明范围,并不限于上述技术特征的特定组合而成的技术方案,同时也应涵盖在不脱离所述发明构思的情况下,由上述技术特征或其等同特征进行任意组合而形成的其它技术方案。例如上述特征与本申请中公开的(但不限于)具有类似功能的技术特征进行互相替换而形成的技术方案。

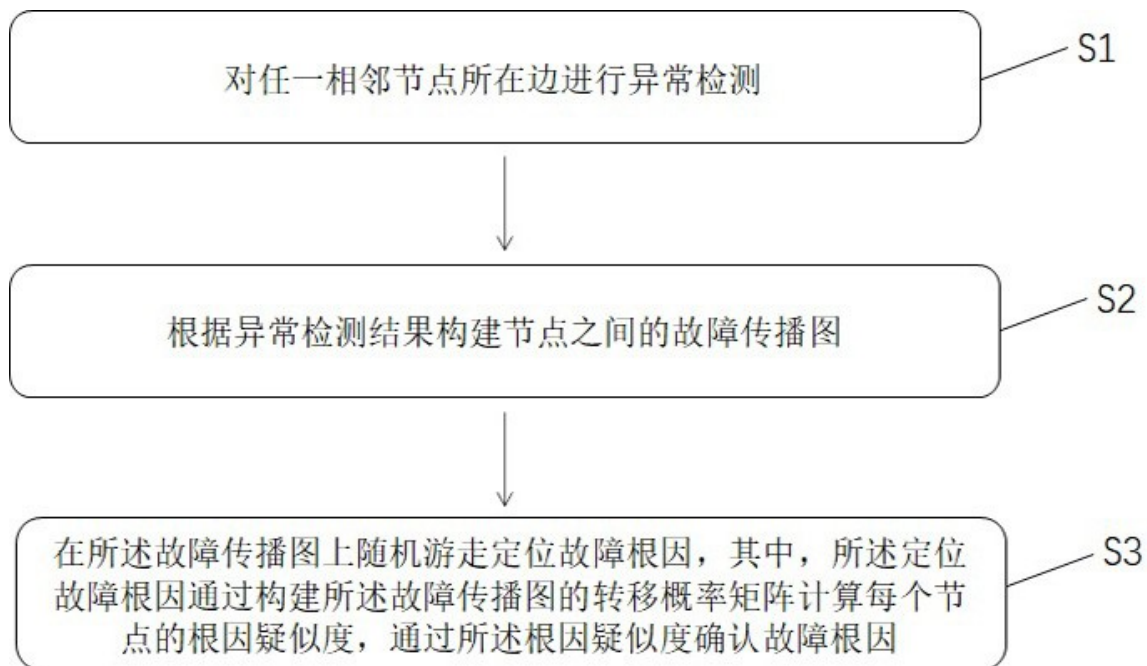


图1

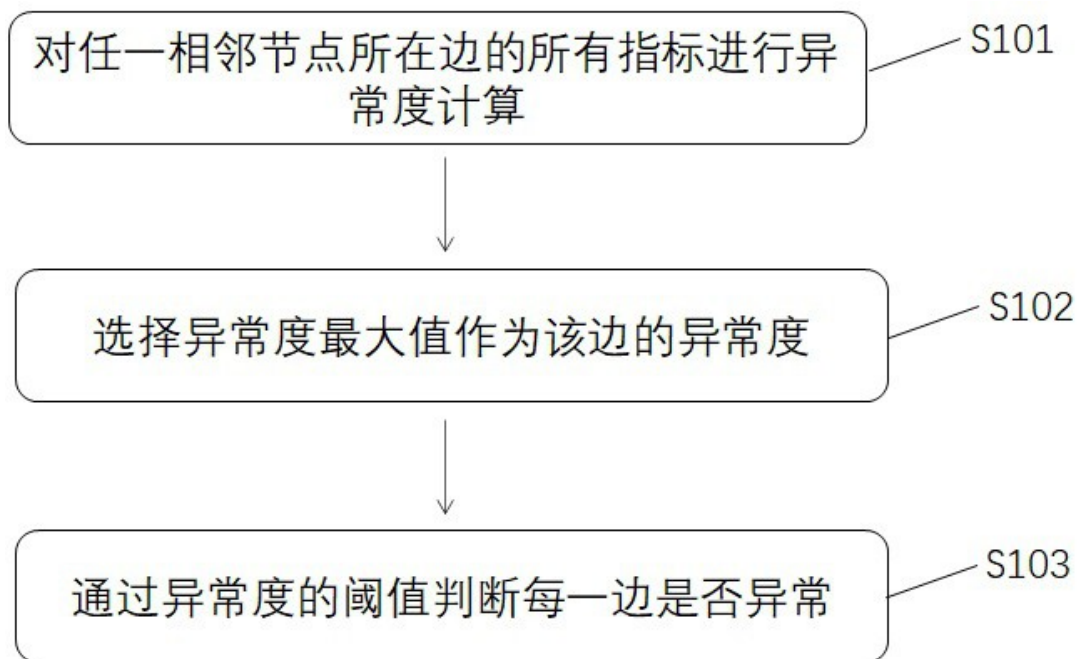


图2

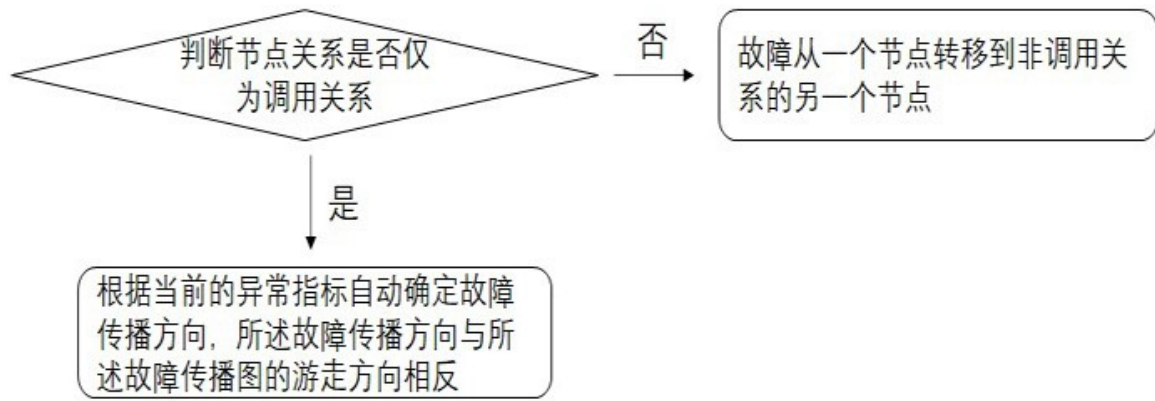


图3



图4



图5

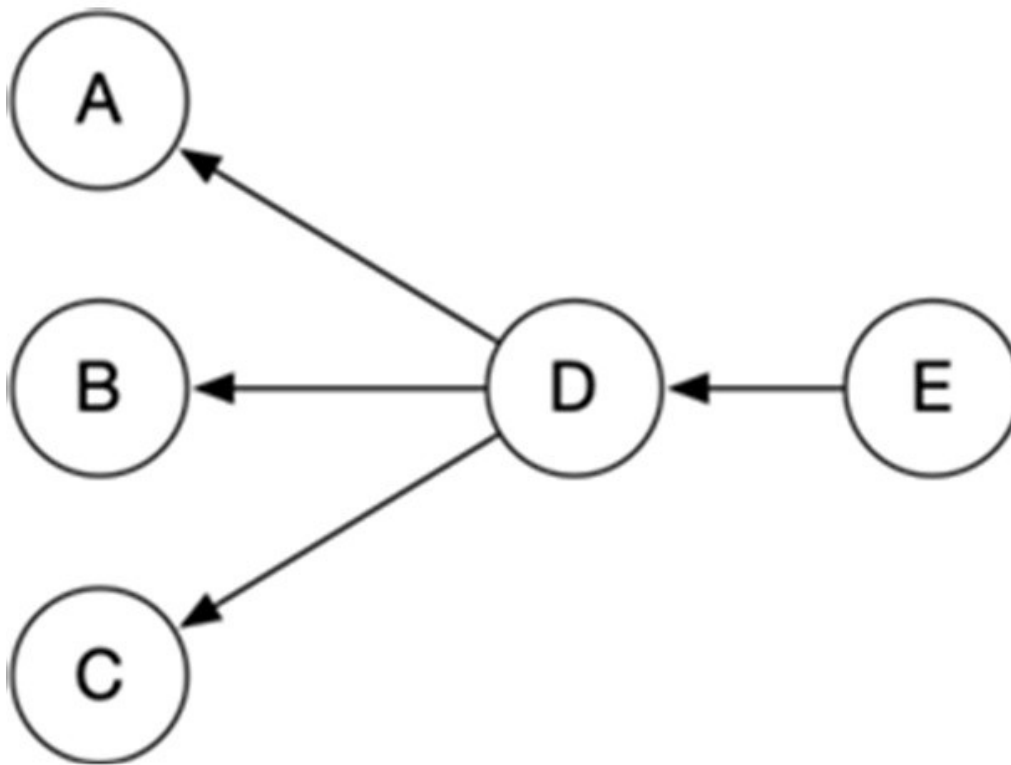


图6

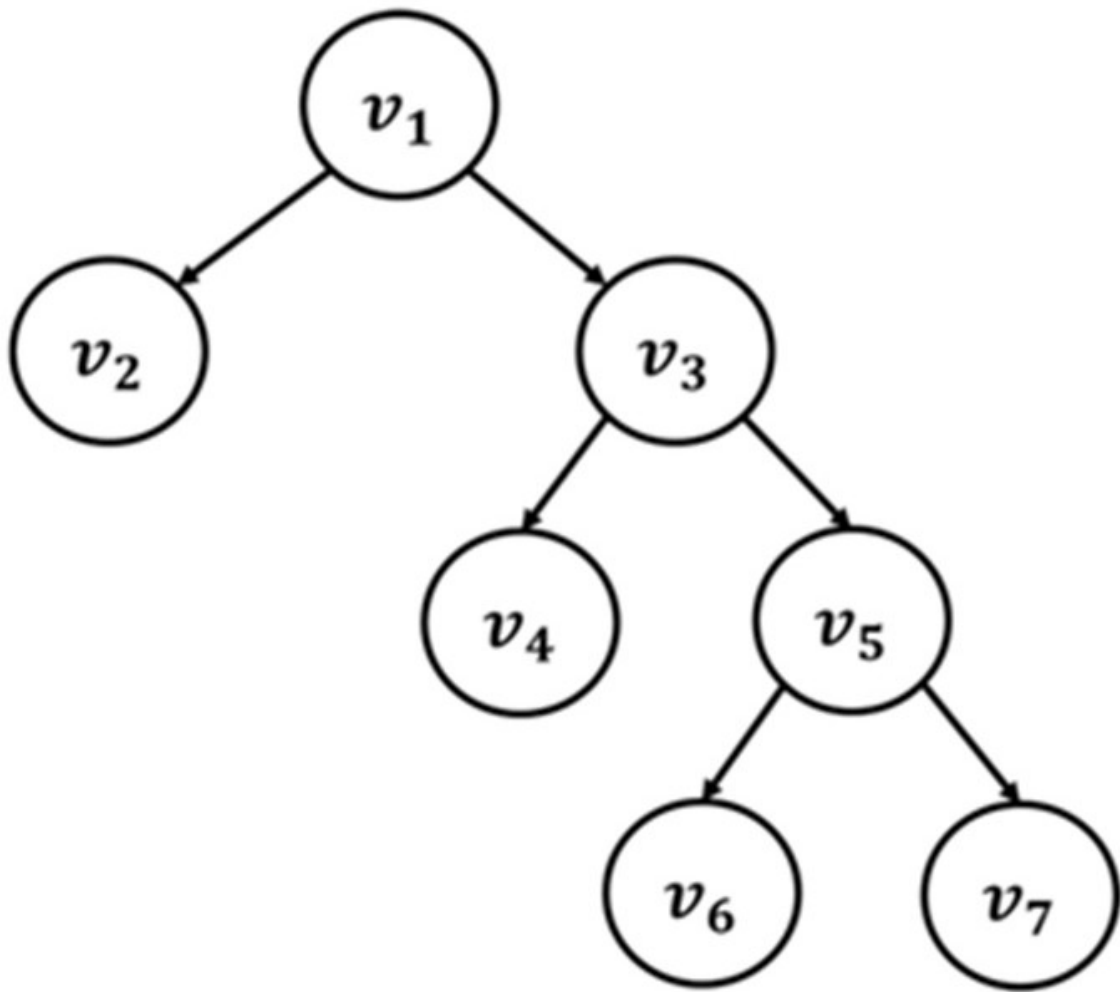


图7